

Міністерство освіти і науки України
Національний Університет “Львівська Політехніка”



Лабораторна робота
з дисципліни
“Об’єктно-орієнтоване програмування, частина 2”

Виконав:
студент гр. ІХ-22
Поліщук Юра

Прийняв:
Гордійчук-Бублівська О.В

Львів-2026

Тема роботи:
Візуалізація даних за допомогою бібліотеки Seaborn

Мета роботи:

Навчитися працювати з бібліотекою Seaborn для візуалізації даних. Ознайомитися з основними типами графіків, які надає бібліотека, навчитися застосовувати їх для аналізу даних.

- Створення теплової карти (Heatmap)
- Створіть теплову карту для аналізу кількості вакансій за рівнем досвіду (Experience Level) та галуззю (Industry).

Підказка: Можна припустити, що кількість вакансій рівна кількості рядків у датафреймі. Тоді скористатися `crosstab()`, що підрахує кількість рядків у датафреймі `df`, які одночасно мають певний рівень досвіду та відносяться до певної галузі:

```
pivot_table = pd.crosstab(df['Experience'], df['Industry'])
```

Або ж використати `groupby()`

- Зробіть висновок про те, де спостерігається найбільша кількість вакансій.
- Створення точкової діаграми (Scatterplot)
- Відобразіть залежність зарплати (Average Salary) від року публікації вакансії (Year, треба витягнути з Date Posted). Точки на графіку пофарбуйте за рівнем досвіду (`hue='Experience Level'`).
- Зробіть висновки коли спостерігається тенденція до зростання зарплат, для вакансій з яким рівнем досвіду.
- Парні графіки (Pairplot)
- Створіть парні графіки для аналізу взаємозв'язків між зарплатою, роком та рівнем досвіду.
- Зробіть висновки з отриманих залежностей та в чому полягає відмінність точкової діаграми від парних графіків.
- Зробіть загальні висновки про роботу

```

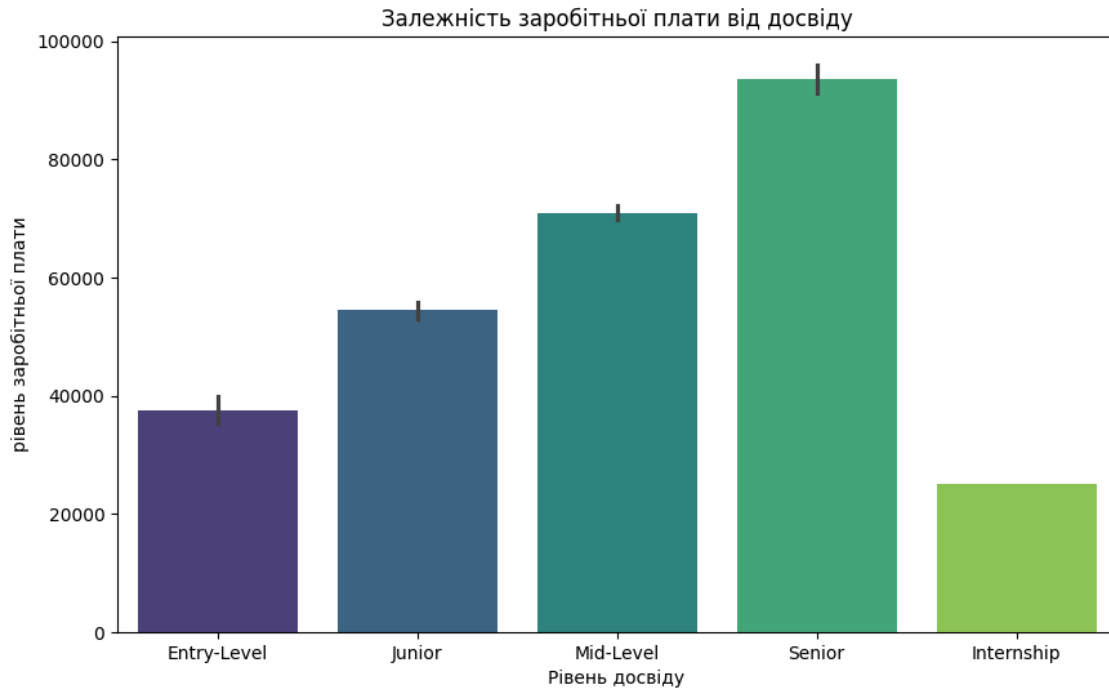
etpy > ...
1  import pandas as pd
2  import seaborn as sns
3  import matplotlib.pyplot as plt
4
5  class CSV:
6      def __init__(self, file: str):
7          self.file=file
8          self.dataset=None
9      def open_csv(self):
10         self.dataset=pd.read_csv(self.file)
11         return self.dataset
12
13 class DataFrame:
14     def __init__(self, df):
15         self.df=df
16     def add_average_salary(self):
17         min_s = self.df['Salary Range'].str.extract(r'£(\d+,?\d*) -')[0]
18         min_s = min_s.str.replace(',', '').astype(int)
19         max_s = self.df['Salary Range'].str.extract(r'£(\d+,?\d*)')[0]
20         max_s = max_s.str.replace(',', '').astype(int)
21         avg_salary = (max_s + min_s) / 2
22         self.df["Average Salary"] = avg_salary
23         return self.df
24     def add_year_col(self):
25         def get_year(date):
26             return pd.to_datetime(date, format='%m/%d/%Y').year
27         self.df['Year'] = self.df['Date Posted'].apply(get_year)
28         return self.df
29
30 class Barplot:
31     def __init__(self, data: pd.DataFrame, x_param: str, y_param: str):
32         self.data=data
33         self.x_param=x_param
34         self.y_param=y_param
35     def plot(self, title, x_label, y_label):
36         try:
37             salary_experience=self.data[[self.x_param, self.y_param]]
38             plt.figure(figsize=(10, 6))
39             sns.barplot(x=self.x_param, y=self.y_param, data=salary_experience, palette='viridis')
40             plt.title(title)
41             plt.xlabel(x_label)
42             plt.ylabel(y_label)
43             plt.show()
44         except KeyError:
45             print("unknown key")
46
47 class Boxplot:
48     def __init__(self, data: pd.DataFrame, x_param: str, y_param: str):
49         self.data=data
50         self.x_param=x_param

```

```

48 class Boxplot:
49     self.y_param=y_param
50     def plot(self, title ,x_label, y_label):
51         try:
52             salary_industry = self.data[[self.x_param, self.y_param]]
53             plt.figure(figsize=(12, 6))
54             plt.xticks(rotation=45, ha='right')
55             plt.tight_layout()
56             sns.boxplot(x=self.x_param, y=self.y_param, data=salary_industry, palette='viridis')
57             plt.title(title)
58             plt.xlabel(x_label)
59             plt.ylabel(y_label)
60             plt.show()
61         except KeyError:
62             print("unknown key")
63         except ValueError:
64             print("value error")
65
66 class Heatmap:
67     def __init__(self, data: pd.DataFrame, x_param: str, y_param: str):
68         self.data = data
69         self.x_param = x_param
70         self.y_param = y_param
71
72     def plot(self, title):
73         try:
74             plt.figure(figsize=(10,6))
75             pivot_table = pd.crosstab(self.data[self.x_param], self.data[self.y_param])
76             sns.heatmap(pivot_table)
77             plt.title(title)
78             plt.show()
79         except KeyError:
80             print("unknown key")
81         except ValueError:
82             print("value error")
83
84 class Scatterplot:
85     def __init__(self, data: pd.DataFrame, x_param: str, y_param: str, hue: str):
86         self.data = data
87         self.x_param = x_param
88         self.y_param = y_param
89         self.hue=hue
90
91     def plot(self, title, x_label, y_label):
92         try:
93             plt.figure(figsize=(10, 6))
94             sns.scatterplot(x=self.x_param, y=self.y_param, hue=self.hue, data=self.data)
95             plt.title(title)
96             plt.xlabel(x_label)
97             plt.ylabel(y_label)
98             plt.show()
99         except KeyError:
100             print("unknown key")
101         except ValueError:
102             print("value error")
103
104 class Pairplot:
105     def __init__(self, data: pd.DataFrame, param_list: list):
106         self.data = data
107         self.param_list=param_list
108
109     def plot(self, title):
110         try:
111             sns.pairplot(self.data[self.param_list], diag_kind='kde', palette='bright')
112             plt.suptitle(title, y=1.02)
113             plt.show()
114         except KeyError:
115             print("unknown key")
116         except ValueError:
117             print("value error")
118
119 ds=CSV("Job opportunities.csv").open_csv()
120
121 dataframe=DataFrame(ds)
122 df_with_avg_salary=dataframe.add_average_salary()
123 df=df_with_avg_salary.add_year_col()
124
125 Barplot(df, "Experience Level", "Average Salary").plot("Залежність заробітної плати від досвіду",
126                                                         "Рівень досвіду", "рівень заробітної плати")
127 Boxplot(df, "Industry", "Average Salary").plot("Діаграма розмаху для відображення розподілу зарплат за галузями",
128                                                  "Галузь", "Середня зоробітна плата")
129 Heatmap(df, "Industry", "Experience Level").plot("Теплова карта для аналізу кількості вакансій за рівнем досвіду та галуззю")
130 Scatterplot(df, "Year", "Average Salary", "Experience Level").plot("залежність зарплати від року публікації",
131                                                                      "Рік публікації", "Середня зарплата")
132 Pairplot(df, ["Average Salary", "Year", "Experience Level"]).plot("Аналіз взаємозв'язків між зарплатою, роком та рівнем досвіду")

```



Висновок

на даній лабораторній роботі я навчився працювати з бібліотекою Seaborn для візуалізації даних. Ознайомитися з основними типами графіків, які надає бібліотека, навчитися застосовувати їх для аналізу даних.